

Smart IT Incident Decision Support System

Complete Line-by-Line Technical Reference Manual: The 6-Phase Pipeline Architecture

An exhaustive, line-by-line pedagogical reference detailing every single module, algorithm, data structure, and anti-leakage safeguard in the ITSM predictive decision support codebase.

Dataset: ServiceNow (141,712 Events / 24,918 Cases)

Architecture: 6 Sequential Phases

Validation: Strict Chronological Split

Models: HistGradientBoosting (GBDT) + Association Rules

Table of Contents

Prelude: Central Configuration (src/config.py)	52 Lines
Phase 1: Cleaning & Dual-Stream Separation (src/p1_preprocess.py)	88 Lines
Phase 2: Process Mining & Bottleneck Discovery (src/p2_process_mining.py)	112 Lines
Phase 3: Case Store, Anomaly Profiling & Clustering (src/p3_profiling_clustering.py)	85 Lines
Phase 4: Root-Cause Pattern & Association Rules (src/p4_association_rules.py)	115 Lines
Phase 5: Temporal Predictive Modeling (ML) (src/p5_predictive_modeling.py)	153 Lines
Phase 6: Operational Dispatcher Dashboard (src/p6_decision_support.py)	94 Lines
Summary: Architectural Integrity & Metrics Matrix	End-to-End

Prelude: Central Configuration (src/config.py)

Defining Global Paths, Column Schemas, and Modeling Hyperparameters (52 Lines)

MODULE MISSION & ARCHITECTURE ROLE

config.py serves as the single source of truth for the entire pipeline. It ensures that file paths, target labels, intake features, and random seeds are never hardcoded across multiple files. If a column name changes or an output directory moves, updating this single file propagates across all 6 phases.

Lines 1–15: Base File Paths & Directory Initialization

```
import os

# Base paths
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_PATH = os.path.join(BASE_DIR, "incident_event_log.csv")
OUTPUT_DIR = os.path.join(BASE_DIR, "outputs")

# Ensure output folder exists
os.makedirs(OUTPUT_DIR, exist_ok=True)
```

Line 1: Imports Python's standard os module for cross-platform file path resolution.

Line 4: BASE_DIR computes the absolute path to the project root directory by navigating two directory levels up from config.py (using dirname(dirname(...))). This guarantees robust path execution regardless of the terminal's working directory.

Lines 5–6: Declares DATA_PATH pointing to the raw incident_event_log.csv and OUTPUT_DIR pointing to the outputs/ folder.

Line 9: os.makedirs(OUTPUT_DIR, exist_ok=True) automatically creates the outputs/ folder on disk if it does not already exist. The parameter exist_ok=True prevents crashes if the directory is already present.

Lines 16–24: Intermediate and Final Artifact Paths

```
CLEANED_TRACES_PATH = os.path.join(OUTPUT_DIR, "cleaned_traces.csv")
ENRICHED_CASES_PATH = os.path.join(OUTPUT_DIR, "enriched_case_features.csv")
OUTLIERS_AUDIT_PATH = os.path.join(OUTPUT_DIR, "pathological_outliers.csv")
DISCOVERED_RULES_PATH = os.path.join(OUTPUT_DIR, "root_cause_rules.csv")
EVALUATION_METRICS_PATH = os.path.join(OUTPUT_DIR, "model_evaluation_metrics.txt")
DECISION_DASHBOARD_PATH = os.path.join(OUTPUT_DIR, "live_dispatcher_queue.csv")
```

Lines 17–23: Declares explicit destination filepaths for all generated outputs across the 6 phases: Stream A traces, Stream B enriched features, Isolation Forest anomaly audit, Association Rules, Machine Learning evaluation benchmarks, and the final dispatcher queue.

Architectural Benefit: Centralizing output paths guarantees that Phase 1 outputs feed directly into Phase 2 and Phase 3 without path mismatch errors.

Lines 25–48: Column Schemas, Intake Features & Ground-Truth Targets

```
ID_COL = "number"
STATE_COL = "incident_state"
GROUP_COL = "assignment_group"
TIMESTAMP_COL = "sys_updated_at"
RESOURCE_COL = "assigned_to"

# Features available strictly at ticket creation (Intake - time zero)
INTAKE_CATEGORICAL = [
    "category", "subcategory", "contact_type", "location",
    "impact", "urgency", "priority"
]

SPARSE_COLS = ["cldb_ci", "problem_id", "rfc", "vendor", "caused_by"]

TARGET_SLA = "made_sla"
TARGET_BREACH = "is_breached"
```

Lines 25–29: Sets aliases for core audit columns: ticket ID (number), workflow state (incident_state), team (assignment_group), and update timestamp (sys_updated_at).

Lines 32–40: INTAKE_CATEGORICAL defines attributes known *strictly* at ticket submission ($t=0$). This list acts as an anti-leakage firewall: no features outside this list are allowed into early breach prediction.

Line 43: SPARSE_COLS lists attributes that are $>98\%$ missing in raw data.

Lines 46–47: Defines the ground-truth prediction targets. made_sla is the raw ServiceNow field (True/False); is_breached is our binary classification target ($\$1 = \text{ext}\{\text{breached}\}\$, \$0 = \text{ext}\{\text{met SLA}\}\$).$

Lines 49–52: Modeling Hyperparameters

```
RANDOM_STATE = 42
TEMPORAL_SPLIT_RATIO = 0.8
OUTLIER_CONTAMINATION = 0.03
```

Line 50: RANDOM_STATE = 42 locks pseudo-random number generation across Isolation Forest and KMeans, ensuring 100% reproducible results across runs.

Line 51: TEMPORAL_SPLIT_RATIO = 0.8 specifies that the oldest 80% of chronological tickets train the models, while the newest 20% evaluate them.

Line 52: OUTLIER_CONTAMINATION = 0.03 instructs Isolation Forest to isolate the top 3% most severe, pathological incident delays.

Phase 1: Data Cleaning & Dual-Stream Separation (src/p1_preprocess.py)

Sanitizing Audit Logs & Separating Event Traces from Intake Snapshots (88 Lines)

PHASE MISSION & WHY DUAL-STREAM IS REQUIRED

A raw ServiceNow log contains multiple chronological rows for each ticket. If you train a machine learning model on raw audit logs, it will accidentally look at future updates (e.g. reassignment count or resolution time), creating severe **future lookahead leakage**. Phase 1 solves this by splitting the data into:

- **Stream A (Event Traces)**: Preserves all update steps to enable process mining.
- **Stream B (Case Intake Matrix)**: Collapses each ticket into exactly 1 row captured at minute zero (\$t=0\$).

Lines 1–13: Module Imports & Loading Raw CSV

```
1: import pandas as pd
2: import numpy as np
3: from src.config import ( DATA_PATH, CLEANED_TRACES_PATH, ID_COL, STATE_COL, GROUP_COL, TIMESTAMP_COL, RE
5: def run_phase1():
...
11:     df = pd.read_csv(DATA_PATH, low_memory=False)
12:     print(f"Raw rows loaded: {len(df):,} | Unique cases: {df[ID_COL].nunique():,}")
```

Lines 1–3: Imports pandas for tabular manipulations, numpy for numerical nulls (`np.nan`), and schema definitions from `config.py`.

Line 5: Declares `run_phase1()` entrypoint function.

Line 11: `pd.read_csv(DATA_PATH, low_memory=False)` loads the 141,712-row dataset. `low_memory=False` forces pandas to parse the whole file in memory without chunk-based type guessing, preventing mixed-type warnings.

Line 12: Prints dataset summary diagnostics: 141,712 event rows across 24,918 unique cases.

Lines 14–26: Cleaning '?' Characters, Filtering State '-100', & Sparse Indicators

```
14: # 1. Clean missing representation: '?' -> NaN
15: df = df.replace("?", np.nan)
17: # 2. Remove corrupted system states (e.g. '-100')
18: initial_len = len(df)
19: df = df[df[STATE_COL] != "-100"].copy()
20: print(f"Filtered {initial_len - len(df)} corrupted rows with state '-100'")
22: # 3. Create binary presence flags for sparse attributes (>98% missing)
23: for col in SPARSE_COLS:
24:     flag_col = f"has_{col}"
25:     df[flag_col] = df[col].notna().astype(int)
```

Line 15: ServiceNow exports missing entries as literal question marks ('?'). `df.replace("?", np.nan)` converts them to standard IEEE floating-point NaN.

Lines 18–20: In ServiceNow, state code "-100" represents corrupted rows from failed background database jobs. Filtering with `df[STATE_COL] != "-100"` purges these invalid rows.

Lines 23–25: Loops over sparse columns (vendor, problem_id, etc.). `df[col].notna().astype(int)` creates 1 if an ID exists or 0 if empty. This preserves the strong signal that a 3rd-party vendor or major problem is involved while discarding hundreds of ungeneralizable text IDs.

Lines 27–34: Timestamp Parsing & Chronological Ordering

```
28: date_cols = ["opened_at", TIMESTAMP_COL, "resolved_at", "closed_at"]
29: for c in date_cols:
30:     df[c] = pd.to_datetime(df[c], format="%d/%m/%Y %H:%M", errors="coerce")
33: df = df.sort_values(by=[ID_COL, TIMESTAMP_COL]).reset_index(drop=True)
```

Lines 28–30: Converts text timestamps into Python `datetime64` objects using the exact ServiceNow format (%d/%m/%Y %H:%M). `errors="coerce"` turns any unparseable strings into NaT rather than crashing.

Line 33: Sorts the table by ticket ID and update timestamp. Chronological sorting is the foundational prerequisite for all process mining in Phase 2.

Lines 35–43: Building Stream A (Event Trace Log)

```
39: trace_cols = [ID_COL, TIMESTAMP_COL, STATE_COL, GROUP_COL, RESOURCE_COL]
40: traces_df = df[trace_cols].copy()
41: traces_df.to_csv(CLEANED_TRACES_PATH, index=False)
42: print(f"Stream A saved to: {CLEANED_TRACES_PATH} ({len(traces_df):,} events)")
```

Lines 39–41: Retains all rows but filters down to the 5 essential process columns: Case ID, Timestamp, State, Assignment Group, and Assigned Technician. Saves to `outputs/cleaned_traces.csv`.

Lines 44–58: Building Stream B (Case Intake Matrix at \$t=0\$)

```
50:     first_updates = df.groupby(ID_COL).first().reset_index()
51:     last_updates = df.groupby(ID_COL).last().reset_index()
53:     cases_df = pd.DataFrame()
54:     cases_df[ID_COL] = first_updates[ID_COL]
55:     cases_df["opened_at"] = first_updates["opened_at"]
56:     cases_df["open_hour"] = first_updates["opened_at"].dt.hour
57:     cases_df["open_dayofweek"] = first_updates["opened_at"].dt.dayofweek
```

Line 50: `df.groupby(ID_COL).first()` captures the first historical record of each ticket—the exact state at minute zero ($t=0$).

Line 51: `df.groupby(ID_COL).last()` captures the final record of each ticket—revealing the ground-truth resolution and whether the SLA was breached.

Lines 56–57: Extracts temporal features: `open_hour` (0–23) and `open_dayofweek` (0=Monday, 6=Sunday). This allows ML models to detect shift change and weekend delays.

Lines 59–88: Target Label Extraction & Execution Guard

```
60:     for col in INTAKE_CATEGORICAL + ["caller_id"]:
61:         cases_df[col] = first_updates[col].fillna("Unknown")
...
70:     final_sla_str = last_updates[TARGET_SLA].astype(str).str.strip().str.lower()
71:     cases_df[TARGET_SLA] = (final_sla_str == "true")
72:     cases_df[TARGET_BREACH] = (~cases_df[TARGET_SLA]).astype(int)
75:     res_time = last_updates["resolved_at"].fillna(last_updates["closed_at"])
76:     cases_df["resolution_time_hrs"] = (res_time - cases_df["opened_at"]).dt.total_seconds() / 3600.0
77:     cases_df["resolution_time_hrs"] = cases_df["resolution_time_hrs"].clip(lower=0.0)
80:     cases_df["final_resolving_group"] = last_updates[GROUP_COL].fillna("Unknown")
85:     return traces_df, cases_df
87: if __name__ == "__main__":
88:     run_phase1()
```

Line 61: Populates static intake attributes from `first_updates` with `fillna("Unknown")`.

Lines 70–72: Standardizes `made_sla` and creates `is_breached` ($\$1 = \text{ext}\{\text{breached}\}\$, \$0 = \text{ext}\{\text{met}\}\$$).

Lines 75–77: Computes resolution duration in hours by subtracting `opened_at` from `res_time`. `.clip(lower=0.0)` eliminates negative clock errors.

Line 80: Extracts the final resolving support team from `last_updates` to serve as the prediction target for smart routing in Phase 5.

Line 85: Returns `(traces_df, cases_df)`.

Lines 87–88: Standard Python entry guard executing `run_phase1()` if called as a standalone script.

Phase 2: Process Mining & Bottleneck Discovery (src/p2_process_mining.py)

Mapping Directly-Follows Graphs, Ping-Pong Rework, and Holding State Dwell Time (112 Lines)

PHASE MISSION & PROCESS MINING MECHANICS

Process mining reconstructs operational reality from audit event timestamps. This phase discovers the transition graph between incident states, detects circular "ping-pong" handoffs between support groups, and quantifies how many hours tickets sit dormant in holding queues (Awaiting Vendor, Awaiting User Info).

Lines 17–31: Function Entrypoint & Chronological Verification

```
17: def run_phase2(traces_df=None):  
...  
22:     if traces_df is None:  
24:         traces_df = pd.read_csv(CLEANED_TRACES_PATH)  
25:         traces_df[TIMESTAMP_COL] = pd.to_datetime(traces_df[TIMESTAMP_COL])  
30:         traces_df = traces_df.sort_values(by=[ID_COL, TIMESTAMP_COL]).copy()
```

Line 17: `run_phase2(traces_df=None)` can accept in-memory traces directly from Phase 1, or load from `cleaned_traces.csv` if run standalone.

Line 30: Re-sorts by ticket ID and timestamp to guarantee strictly ascending order before applying window shift operations.

Lines 32–42: Directly-Follows Graph (DFG) Step Duration Calculation

```
33:     traces_df["next_state"] = traces_df.groupby(ID_COL)[STATE_COL].shift(-1)  
34:     traces_df["next_group"] = traces_df.groupby(ID_COL)[GROUP_COL].shift(-1)  
35:     traces_df["next_timestamp"] = traces_df.groupby(ID_COL)[TIMESTAMP_COL].shift(-1)  
38:     traces_df["step_duration_hrs"] = (  
39:         (traces_df["next_timestamp"] - traces_df[TIMESTAMP_COL]).dt.total_seconds() / 3600.0  
40:     ).clip(lower=0.0)
```

Lines 33–35: `.shift(-1)` looks ahead by exactly one row *within each ticket group* to determine: What was the next state? What was the next group? When did that next transition happen?

Lines 38–40: Subtracts the current timestamp from the next timestamp to calculate the exact duration (in hours) spent in the current state before transitioning.

Lines 43–61: Aggregating Transitions & Exporting Bottlenecks CSV

```
43:     state_transitions = (  
44:         traces_df.dropna(subset=["next_state"])  
45:         .groupby([STATE_COL, "next_state"])  
46:         .agg(  
47:             frequency=(ID_COL, "count"),  
48:             avg_duration_hrs=("step_duration_hrs", "mean")  
49:         )  
50:         .reset_index()  
51:         .sort_values(by="frequency", ascending=False)  
52:     )  
58:     bottlenecks_path = os.path.join(OUTPUT_DIR, "process_bottlenecks.csv")  
59:     state_transitions.to_csv(bottlenecks_path, index=False)
```

Lines 43–52: Aggregates state pairs into a transition matrix. For every state hop (e.g. Active -> Resolved or Awaiting Vendor -> Awaiting Vendor), it calculates total frequency and average hours spent.

Line 59: Exports the transition matrix to outputs/process_bottlenecks.csv.

Lines 62–74: Ping-Pong Rework Loop Detection

```
65:     traces_df["prev_group"] = traces_df.groupby(ID_COL)[GROUP_COL].shift(1)  
66:     traces_df["prev_prev_group"] = traces_df.groupby(ID_COL)[GROUP_COL].shift(2)  
68:     traces_df["is_ping_pong_step"] = (  
69:         (traces_df[GROUP_COL] == traces_df["prev_prev_group"]) &  
70:         (traces_df[GROUP_COL] != traces_df["prev_group"]) &  
71:         traces_df[GROUP_COL].notna() &  
72:         traces_df["prev_prev_group"].notna()  
73:     ).astype(int)
```

Lines 65–66: Shifts backward by 1 step (prev_group) and 2 steps (prev_prev_group).

Lines 68–73: Identifies a ping-pong bounce: when the current team equals the team from two steps ago, but is different from the team one step ago (\$ ext{Group A} ightarrow ext{Group B} ightarrow ext{Group A}\$). Marks the event with a binary flag (1).

Why this is critical: Ping-pong bouncing is the single greatest multiplier of customer dissatisfaction and SLA failure. Detecting it pinpoints teams with ambiguous service boundaries.

Lines 75–90: Quantifying Holding State Dwell Times

```
77:     waiting_states = ["Awaiting User Info", "Awaiting Vendor", "Awaiting Problem", "Awaiting Evidence"]
78:     traces_df["is_waiting_state"] = traces_df[STATE_COL].isin(waiting_states).astype(int)
79:     traces_df["waiting_dwell_hrs"] = traces_df["step_duration_hrs"] * traces_df["is_waiting_state"]
82:     traces_df["is_handoff"] = (
83:         traces_df["next_group"].notna() &
84:         (traces_df[GROUP_COL] != traces_df["next_group"])
85:     ).astype(int)
87:     traces_df["active_dwell_hrs"] = (
88:         traces_df["step_duration_hrs"] * (traces_df[STATE_COL] == "Active")
89:     ).fillna(0.0)
```

Lines 77–79: Flags tickets in non-active holding states. Multiplies `step_duration_hrs` by the flag to calculate exact dormant hours spent waiting.

Lines 82–85: Checks if the next group is different from the current group, marking a team handoff (`is_handoff = 1`).

Lines 87–89: Calculates hours spent in the Active state where work was actively being performed.

Lines 91–112: Per-Case Behavioral Aggregation, Print & Execution Guard

```
93:     process_features = traces_df.groupby(ID_COL).agg(
94:         total_events=(STATE_COL, "count"),
95:         unique_groups=(GROUP_COL, "nunique"),
96:         total_handoffs=("is_handoff", "sum"),
97:         ping_pong_count=("is_ping_pong_step", "sum"),
98:         total_wait_hours=("waiting_dwell_hrs", "sum"),
99:         total_active_hours=("active_dwell_hrs", "sum")
100:     ).reset_index()
102:     process_features["is_ping_pong"] = (process_features["ping_pong_count"] > 0).astype(int)
109:     return process_features
111: if __name__ == "__main__":
112:     run_phase2()
```

Lines 93–100: Aggregates the multi-row event traces into exactly 1 summary row per ticket ID. Computes: total events, distinct groups touched, handoff count, ping-pong bounces, total waiting hours, and active work hours.

Line 102: Creates a binary flag `is_ping_pong = 1` if the ticket experienced any ping-pong rework.

Line 109: Returns the aggregated DataFrame `process_features`.

Lines 111–112: Main execution guard.

Phase 3: Case Feature Store, Anomaly Profiling & Workload Clustering (src/p3_profiling_clustering.py)

Unified Feature Fusion, Isolation Forest Outlier Profiling, & K-Means Archetypes (85 Lines)

PHASE MISSION & THE RETENTION RULE

This phase fuses Stream B intake attributes with Phase 2 process metrics into a single table. It runs an **Isolation Forest** to identify the top 3% most severe, pathological incidents.

Crucial Rule: Unlike standard ML pipelines that discard outliers, in ITSM, outliers are catastrophic outages! They are **audited and retained** to ensure our root-cause models study them. It also applies **K-Means** to segment tickets into 3 operational workload types.

Lines 18–27: Feature Fusion into Unified Case Store

```
18: def run_phase3(cases_df, process_features):  
...  
25:     enriched_df = pd.merge(cases_df, process_features, on=ID_COL, how="inner")  
26:     print(f"Unified Case Feature Store created: {len(enriched_df):,} rows x {len(enriched_df.columns)}")
```

Line 25: Performs an inner join on ID_COL (ticket number) between cases_df (24,918 intake rows) and process_features (24,918 process metric rows). The resulting enriched_df contains 30 features per incident.

Lines 28–60: Pathological Outlier Profiling via Isolation Forest

```
31:     anomaly_feature_cols = [  
32:         "resolution_time_hrs", "total_events", "total_handoffs",  
33:         "total_wait_hours", "ping_pong_count"  
34:     ]  
37:     X_anom = enriched_df[anomaly_feature_cols].fillna(0).values  
39:     scaler = StandardScaler()  
40:     X_anom_scaled = scaler.fit_transform(X_anom)  
42:     iso_forest = IsolationForest(  
43:         contamination=OUTLIER_CONTAMINATION,  
44:         random_state=RANDOM_STATE,  
45:         n_estimators=100  
46:     )  
48:     preds = iso_forest.fit_predict(X_anom_scaled)  
49:     enriched_df["is_pathological"] = (preds == -1).astype(int)  
51:     outliers_df = enriched_df[enriched_df["is_pathological"] == 1].copy()  
52:     outliers_df.to_csv(OUTLIERS_AUDIT_PATH, index=False)
```

Lines 31–34: Selects the 5 dimensions of operational extremity: resolution duration, audit events, team handoffs, wait time, and ping-pong bounces.

Lines 39–40: `StandardScaler` centers each feature around zero mean with unit variance so large numbers (e.g. 500 hours) do not overpower small numbers (e.g. 4 handoffs).

Lines 42–46: Initializes an `IsolationForest` with 100 trees and 3% contamination. Isolation Forest isolates anomalies by randomly partitioning feature space; extreme cases require fewer splits to isolate.

Lines 48–49: Outliers receive `preds == -1`, which we convert to a binary flag `is_pathological = 1`.

Lines 51–52: Extracts all 748 pathological cases and saves them to `outputs/pathological_outliers.csv` for executive governance.

Lines 61–85: Workload Segmentation via K-Means, Export & Execution Guard

```
63:     cluster_features = ["resolution_time_hrs", "total_handoffs", "total_wait_hours"]  
64:     X_clust = scaler.fit_transform(enriched_df[cluster_features].fillna(0).values)  
66:     kmeans = KMeans(n_clusters=3, random_state=RANDOM_STATE, n_init=10)  
67:     enriched_df["workload_cluster"] = kmeans.fit_predict(X_clust)  
75:     enriched_df.to_csv(ENRICHED_CASES_PATH, index=False)  
78:     return enriched_df  
80: if __name__ == "__main__":  
...  
85:     run_phase3(cases, feats)
```

Lines 63–67: Applies **K-Means clustering** with $k=3$ on scaled resolution time, handoffs, and waiting hours. Discovers 3 natural operational workload archetypes: (Cluster 0: Fast-track quick fixes; Cluster 1: Multi-hop technical escalations; Cluster 2: Severe vendor/holding delays).

Line 75: Writes the complete feature store to `outputs/enriched_case_features.csv`.

Line 78: Returns `enriched_df`.

Lines 80–85: Test block allowing independent execution of Phase 3.

Phase 4: Root-Cause Pattern & Association Rule Mining (src/p4_association_rules.py)

Mining Human-Readable IF-THEN Rules with Support, Confidence, and Lift (115 Lines)

PHASE MISSION & SYMBOLIC EXPLAINABILITY

Machine learning models are often "black boxes" that output probability scores without explanation. Phase 4 solves this by applying **Association Rule Mining** to discover symbolic IF-THEN rules (e.g., *IF {Priority: 2 - High} THEN {Outcome: SLA Breached} with 97.1% Confidence and 2.65x Lift*). These rules provide transparent justifications for the dispatcher dashboard in Phase 6.

Lines 18–47: Continuous Metric Discretization into Itemsets

```
19: trans_df = pd.DataFrame()
22: trans_df["Handoffs"] = pd.cut(
23:     enriched_df["total_handoffs"],
24:     bins=[-1, 1, 3, 100],
25:     labels=["Handoffs: 0-1", "Handoffs: 2-3", "Handoffs: 4+"]
26: ).astype(str)
29: trans_df["WaitTime"] = pd.cut(
30:     enriched_df["total_wait_hours"],
31:     bins=[-1, 2, 24, 10000],
32:     labels=["Wait: Low (<2h)", "Wait: Med (2-24h)", "Wait: High (>24h)"]
33: ).astype(str)
36: trans_df["PingPong"] = enriched_df["is_ping_pong"].map({1: "PingPong: Yes", 0: "PingPong: No"})
39: trans_df["Priority"] = "Priority: " + enriched_df["priority"].astype(str)
43: top_cats = enriched_df["category"].value_counts().head(8).index
44: trans_df["Category"] = enriched_df["category"].apply(lambda c: f"Cate: {c}" if c in top_cats else "C
46: trans_df["Target"] = enriched_df[TARGET_BREACH].map({1: "Outcome: SLA Breached", 0: "Outcome: SLA Met"}
```

Lines 22–33: Association rule mining requires discrete symbolic items. `pd.cut()` converts continuous numbers into categorical bins (e.g. Handoffs: 4+ or Wait: High (>24h)).

Lines 36–44: Encodes ping-pong status, prefixes priority names, and retains the top 8 categories while grouping rare categories into Cate: Other to prevent rare itemset combinatorial explosion.

Line 46: Creates the rule consequent: Outcome: SLA Breached vs Outcome: SLA Met.

Lines 48–78: Combinatorial Mining of 1-Item and 2-Item Antecedents

```
48:     n_cases = len(trans_df)
49:     target_val = "Outcome: SLA Breached"
50:     base_support_target = (trans_df["Target"] == target_val).mean()
54:     feature_cols = ["Category", "Priority", "Handoffs", "WaitTime", "PingPong"]
59:     for r in [1, 2]:
60:         for col_combo in itertools.combinations(feature_cols, r):
61:             grouped = trans_df.groupby(list(col_combo))
62:             for val_combo, group in grouped:
63:                 ant_count = len(group)
64:                 if ant_count < 50: continue
68:                 support_ant = ant_count / n_cases
71:                 both_count = (group["Target"] == target_val).sum()
72:                 support_both = both_count / n_cases
75:                 confidence = both_count / ant_count if ant_count > 0 else 0
78:                 lift = confidence / base_support_target if base_support_target > 0 else 0
```

Line 50: Computes the global baseline SLA breach probability ($\approx 36.6\%$).

Lines 59–60: Uses `itertools.combinations` to systematically evaluate all 1-item antecedents (e.g. Priority) and 2-item antecedents (e.g. WaitTime AND PingPong).

Lines 63–64: Prunes infrequent combinations with < 50 occurrences to eliminate statistical noise.

Lines 75–78: Computes standard association rule metrics:

- **Support:** Frequency of co-occurrence: $\text{Support}(A \cap B) = \frac{\text{Count}(A \cap B)}{N}$.
- **Confidence:** Conditional probability: $P(B \mid A) = \frac{\text{Count}(A \cap B)}{\text{Count}(A)}$.
- **Lift:** Multiplier over baseline: $\text{Lift} = \frac{\text{Confidence}}{P(B)}$. A lift of 2.5 times means the pattern makes SLA breach 2.5 times more likely than normal.

Lines 79–115: Threshold Filtering, Rule Export & Execution Guard

```
81:         if confidence >= 0.60 and lift > 1.25 and support_both >= 0.01:
82:             ant_str = " AND ".join(val_combo) if isinstance(val_combo, tuple) else val_combo
83:             rules_list.append({
84:                 "Antecedent": ant_str, "Consequent": target_val,
85:                 "Support": round(support_both, 4), "Confidence": round(confidence, 4),
86:                 "Lift": round(lift, 2), "Frequency": both_count
87:             })
92:     rules_df = pd.DataFrame(rules_list)
94:     rules_df = rules_df.sort_values(by=["Confidence", "Lift"], ascending=False).reset_index(drop=True)
95:     rules_df.to_csv(DISCOVERED_RULES_PATH, index=False)
106:     return rules_df
108: if __name__ == "__main__":
...
115:     run_phase4(enriched)
```

Line 81: Enforces strict filtering thresholds: minimum 60% confidence, minimum 1.25 times positive lift, and at least 1% dataset support.

Lines 94–95: Sorts rules from highest to lowest confidence and saves the top 28 high-impact rules to `outputs/root_cause_rules.csv`.

Line 106: Returns `rules_df`.

Lines 108–115: Main execution guard.

Phase 5: Multi-Stage Supervised Predictive Analytics (src/p5_predictive_modeling.py)

Temporal Split, HistGradientBoosting Ensemble, & Model Serialization (153 Lines)

PHASE MISSION & TEMPORAL VALIDATION STRATEGY

This phase builds two supervised machine learning models using **Gradient Boosted Decision Trees (GBDT)**:

1. **Model 1 (SLA Breach Classifier)**: Predicts probability of failure at ticket intake (\$t=0\$).
2. **Model 2 (Smart Routing Recommender)**: Predicts the exact resolving support group.

Anti-Leakage Firewall: To mimic real production deployment, we enforce a strict **chronological split** (older 80% train, newer 20% test). Standard random K-fold CV is strictly forbidden because it leaks future organizational state into past predictions.

Lines 25–36: Chronological Ordering & Strict Temporal Train/Test Split

```
26:     enriched_df = enriched_df.sort_values(by="opened_at").reset_index(drop=True)
27:     n_total = len(enriched_df)
28:     n_train = int(n_total * TEMPORAL_SPLIT_RATIO)
30:     train_df = enriched_df.iloc[:n_train].copy()
31:     test_df = enriched_df.iloc[n_train:].copy()
```

Line 26: Sorts all 24,918 incidents strictly by opened_at date.

Lines 28–31: Splits the data chronologically: the first 19,934 tickets (February–June 2016) form the **Train Set**, while the final 4,984 tickets (July–December 2016) form the **Unseen Test Set**.

Lines 37–56: Feature Selection & Categorical Ordinal Encoding

```
38:     intake_features = ["open_hour", "open_dayofweek"] + INTAKE_CATEGORICAL + [f"has_{c}" for c in SPARS
41:     cat_cols_indices = [i for i, c in enumerate(intake_features) if c in INTAKE_CATEGORICAL]
43:     encoder = OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1)
45:     X_train_cat = encoder.fit_transform(train_df[INTAKE_CATEGORICAL].astype(str))
46:     X_test_cat = encoder.transform(test_df[INTAKE_CATEGORICAL].astype(str))
48:     X_train = train_df[intake_features].copy()
49:     X_test = test_df[intake_features].copy()
51:     X_train[INTAKE_CATEGORICAL] = X_train_cat
52:     X_test[INTAKE_CATEGORICAL] = X_test_cat
54:     y_train = train_df[TARGET_BREACH].values
55:     y_test = test_df[TARGET_BREACH].values
```

Line 38: Assembles the intake feature vector: creation hour, weekday, static categories (category, priority, etc.), and binary sparse flags. Zero process metrics are included!

Lines 43–46: Fits OrdinalEncoder on training data only. handle_unknown="use_encoded_value", unknown_value=-1 ensures that if an unseen category appears in future test tickets, the pipeline maps it to -1 without crashing.

Lines 57–79: Training Model 1 (Intake SLA Breach Classifier)

```
61:     clf_breach = HistGradientBoostingClassifier(  
62:         categorical_features=cat_cols_indices,  
63:         random_state=RANDOM_STATE,  
64:         max_iter=150, learning_rate=0.08, min_samples_leaf=25  
65:     )  
68:     clf_breach.fit(X_train, y_train)  
70:     y_pred_proba = clf_breach.predict_proba(X_test)[:, 1]  
71:     y_pred = (y_pred_proba >= 0.5).astype(int)  
73:     auc = roc_auc_score(y_test, y_pred_proba)  
74:     report = classification_report(y_test, y_pred, target_names=["Met SLA", "Breached SLA"])
```

Lines 61–68: Initializes and trains a `HistGradientBoostingClassifier` (scikit-learn's optimized GBDT algorithm). It handles categorical features natively using integer bins. Hyperparameters: 150 iterations, learning rate 0.08, and minimum leaf size 25 to prevent overfitting.

Lines 70–74: Predicts breach probabilities on the future test set. Evaluates performance using **ROC-AUC** (achieving **0.7594**) and generates precision/recall metrics.

Lines 80–105: Training Model 2 (Smart Routing Recommender)

```
85:     top_groups = train_df["final_resolving_group"].value_counts().head(12).index.tolist()  
88:     train_mask = train_df["final_resolving_group"].isin(top_groups)  
89:     test_mask = test_df["final_resolving_group"].isin(top_groups)  
91:     group_encoder = OrdinalEncoder()  
92:     y_train_grp = group_encoder.fit_transform(train_df.loc[train_mask, ["final_resolving_group"]]).ravel()  
93:     y_test_grp = group_encoder.transform(test_df.loc[test_mask, ["final_resolving_group"]]).ravel()  
95:     clf_route = HistGradientBoostingClassifier(  
96:         categorical_features=cat_cols_indices, random_state=RANDOM_STATE, max_iter=100  
97:     )  
100:     clf_route.fit(X_train.loc[train_mask], y_train_grp)  
101:     y_pred_grp = clf_route.predict(X_test.loc[test_mask])  
103:     routing_acc = accuracy_score(y_test_grp, y_pred_grp)
```

Line 85: Identifies the top 12 resolving specialist groups handling the bulk of corporate incidents.

Lines 91–93: Encodes group names into numeric class indices.

Lines 95–103: Trains Model 2 to predict the resolving group directly from ticket intake attributes. Evaluates Top-1 accuracy, achieving **75.7% accuracy** across unseen future incidents.

Lines 106–153: Evaluation Report, Serialization & Main Guard

```
107:     with open(EVALUATION_METRICS_PATH, "w", encoding="utf-8") as f:
108:         f.write("=== SMART IT INCIDENT DECISION SUPPORT - MODEL EVALUATION ===")
109:
110:     """
111:     ...
122:     test_df["predicted_breach_risk"] = y_pred_proba
123:     test_df["predicted_breach_flag"] = y_pred
127:     test_df["recommended_group"] = group_encoder.inverse_transform(test_pred_grp_indices.reshape(-1, 1))
130:     import joblib
131:     models_path = os.path.join(OUTPUT_DIR, "trained_models.joblib")
132:     model_artifacts = {
133:         "clf_breach": clf_breach, "clf_route": clf_route,
134:         "encoder": encoder, "group_encoder": group_encoder,
135:         "intake_features": intake_features, "INTAKE_CATEGORICAL": INTAKE_CATEGORICAL,
136:         "SPARSE_COLS": SPARSE_COLS
137:     }
141:     joblib.dump(model_artifacts, models_path)
144:     return clf_breach, clf_route, test_df
146: if __name__ == "__main__":
147:     ...
153:     run_phase5(enriched)
```

Lines 107–119: Exports formal model validation metrics to `outputs/model_evaluation_metrics.txt`.

Lines 122–127: Appends predicted risk probability and recommended group to `test_df` for Phase 6.

Lines 130–141: Bundles trained classifiers, encoders, and feature column lists into a single Python dictionary and saves to `outputs/trained_models.joblib` via `joblib.dump()` for instant standalone scoring.

Line 144: Returns (`clf_breach`, `clf_route`, `test_df`).

Lines 146–153: Main execution guard.

Phase 6: Operational Decision Support Dashboard (src/p6_decision_support.py)

Risk Tiers, Prescribed Actions, Rule-Based Explainability & Dispatcher Queue (94 Lines)

PHASE MISSION & THE OPERATIONAL DISPATCHER EXPERIENCE

This phase bridges machine learning and human operations. It translates the raw mathematical risk score (\$0.0\$ to \$1.0\$) into operational priority tiers (**HIGH RISK**, **MEDIUM**, **LOW RISK**), assigns an exact recommended routing action, pairs the ticket with a human-readable root-cause explanation from Phase 4, and exports the live dispatcher queue to outputs/live_dispatcher_queue.csv.

Lines 18–38: Risk Tiering & Prescriptive Action Logic

```
18:     dashboard_records = []
19:     for _, row in test_df.iterrows():
20:         ...
21:         # Determine Risk Tier
22:         if risk_score >= 0.70:
23:             tier = "HIGH RISK"
24:             action = f"URGENT: Bypass L1 -> Dispatch direct to {rec_group}"
25:         elif risk_score >= 0.40:
26:             tier = "MEDIUM"
27:             action = f"MONITOR: Route to {rec_group} with 24h milestone alert"
28:         else:
29:             tier = "LOW RISK"
30:             action = f"STANDARD: Route to {rec_group} (Fast-track)"
```

Lines 28–37: Applies operational thresholds to incoming tickets:

- **\$\ge 70\%\$ (HIGH RISK):** Prescribes immediate Level 1 triage bypass, routing straight to the specialist team.
- **\$40\% - 69\%\$ (MEDIUM):** Routes to specialist with proactive 24-hour SLA milestone alerts.
- **\$< 40\%\$ (LOW RISK):** Routes normally; eligible for fast-track self-service resolution.

Lines 39–57: Symbolic Explainability Rule Matching

```
39:         # Match with discovered rules for explainability
40:         explanation = "Standard ticket characteristics."
41:         if not rules_df.empty:
42:             cat_match = rules_df[rules_df["Antecedent"].str.contains(str(cat), na=False)]
43:             if not cat_match.empty:
44:                 top_rule = cat_match.iloc[0]
45:                 explanation = f"Matches Rule: {{{top_rule['Antecedent']}}} (Historical Confidence: {top_rule['Confidence']})"
46:             elif risk_score >= 0.70:
47:                 explanation = f"High risk pattern: {cat} with {pri} priority."
48:             else:
49:                 explanation = "No rule matched the category."
50:         dashboard_records.append({
51:             "Ticket_ID": ticket_id, "Category": cat, "Priority": pri,
52:             "Breach_Risk": f"{risk_score*100:.1f}%", "Risk_Tier": tier,
53:             "Recommended_Action": action, "Explainable_Reason": explanation
54:         })
```

Lines 39–48: Matches the ticket against Phase 4 association rules. If a rule matched the category, it quotes the exact antecedent condition and historical confidence percentage. This eliminates the "black-box" dilemma for human managers.

Lines 50–54: Packages the ticket into a complete Decision Card dictionary.

Lines 59–94: Queue Export, Live Preview & Main Guard

```
59:     queue_df = pd.DataFrame(dashboard_records)
60:     queue_df = queue_df.sort_values(by="Risk_Tier", ascending=True).reset_index(drop=True)
61:     queue_df.to_csv(DECISION_DASHBOARD_PATH, index=False)
62:     sample_high = queue_df[queue_df["Risk_Tier"] == "HIGH RISK"].head(4)
63:     sample_med = queue_df[queue_df["Risk_Tier"] == "MEDIUM"].head(3)
64:     sample_low = queue_df[queue_df["Risk_Tier"] == "LOW RISK"].head(3)
65:     sample_display = pd.concat([sample_high, sample_med, sample_low])
66:     ...
67:     return queue_df
68: if __name__ == "__main__":
69:     ...
70:     run_phase6(test_df, rules)
```

Lines 59–61: Creates the final queue_df sorted by priority and exports to outputs/live_dispatcher_queue.csv.

Lines 66–79: Formats and prints a live preview of the dispatcher dashboard across High, Medium, and Low risk tiers to the terminal.

Line 81: Returns queue_df.

Lines 83–94: Main execution guard executing all phases end-to-end if run directly.

SUMMARY: END-TO-END ARCHITECTURAL INTEGRITY CHECKLIST

Phase	Key Input	Key Output	Core Technical Safeguard
Phase 1	Raw CSV (141,712 rows)	Stream A traces & Stream B intake	Dual-stream separation prevents future data leakage (\$t=0\$).
Phase 2	Stream A Traces	Bottlenecks CSV & Process features	Shift windowing identifies ping-pong rework & idle dwell time.
Phase 3	Intake + Process features	Enriched store & Outlier audit	Isolation Forest retains (never drops!) catastrophic incidents.
Phase 4	Enriched case store	28 Root-cause symbolic rules	Discretization + Support/Confidence/Lift mining for explainability.
Phase 5	Enriched case store	Trained GBDT models & Joblib	Strict chronological 80/20 train-test split (ROC-AUC 0.7594).
Phase 6	Test predictions + Rules	Live dispatcher queue CSV	Synthesizes numerical probability with symbolic IF-THEN justifications.